Average

final : 19.5 out of 32

Machine Learning Assignment 3

Simon Cooper

P1: 2/4

P3(a,d1): 7.5/12

I, Simon Cooper, pledge that this assignment is completely my own work, and that I did not take, borrow or steal work from any other person, and that I did not allow any other person to use, have, borrow or steal portions of my work. I understand that if I violate this honesty pledge, I am subject to disciplinary action pursuant to the appropriate sections of Imperial College London.

1

The equation can be expanded as below, where $h^* = min_{h \in \mathcal{H}} R(h)$:

$$R(h_\epsilon) - R(h^*) = R(h_\epsilon) - \hat{R}_n(h_\epsilon) + \hat{R}_n(h_\epsilon) - \hat{R}_n(h^*) + \hat{R}_n(h^*) - R(h^*)$$

As g and h^* are in the set $\mathcal{H}$, and for all $h \in \mathcal{H}$:

$$|\hat{R}_n(h) - R(h)| \leq \sqrt{\frac{log\frac{2M}{\delta}}{2n}}$$

And as $\hat{R}_n(h_\epsilon)$ *must* be no larger than $\hat{R}_n(h^*)$, as the latter is optimal:

$$\hat{R}_n(h_\epsilon) - \hat{R}_n(h^*) \leq 0$$

Therefore:

$$(R(h_\epsilon) - \hat{R}_n(h_\epsilon)) + (\hat{R}_n(h_\epsilon) - \hat{R}_n(h^*)) + (\hat{R}_n(h^*) - R(h^*)) \leq 0 + 2\sqrt{\frac{log\frac{2M}{\delta}}{2n}}$$

$$R(h_\epsilon) - R(h^*) \leq \sqrt{4\frac{log\frac{2M}{\delta}}{2n}}$$

The below is true with probability $1 - \delta$, and is the bound

$$R(h_\epsilon) - R(h^*) \leq \sqrt{\frac{2log\frac{2M}{\delta}}{n}}$$

2

The error is effectively a normalised sum of the distance of every y_i from a point m. This point is selected as the median value of all y_i, as the median point has the lowest combined distance to each other point in a set.
Imagining n data points on a line, as seen in Figure 1, with the initial value of m to be below that of the smallest y. Error is:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - m|$$

As m is below the smallest y, the above equation equals:

$$\frac{1}{n}\sum_{i=1}^{n} y_i - m$$

Increasing m by δ (Figure 2) results in a decrease in error of δ overall, as seen below:

$$\frac{1}{n}\sum_{i=1}^{n} y_i - (m + \delta)$$



Figure 1: One dimensional features y_i, with point m



Figure 2: Increasing m by δ

$$\frac{1}{n}\left(\sum_{i=1}^{n} y_i - m\right) - \frac{n}{n}\delta$$

Now imagine m is incremented so that it equals the smallest y. Now when m is incremented further, it will *increase* in distance from the smallest point. However it will continue to get closer to the remaining $n-1$ points, so therefore an increase in m by δ will result in:

$$\frac{1}{n}\left(\sum_{i=1}^{n-1} (y_i - (m+\delta)) + (m+\delta) - y_{min}\right)$$

Taking out δ:

$$\frac{1}{n}\left(\sum_{i=1}^{n-1} (y_i - m) + m - y_{min}\right) - \frac{(n-1)\delta - \delta}{n}$$

$$\frac{1}{n}\left(\sum_{i=1}^{n-1} (y_i - m) + m - y_{min}\right) - \frac{n-2}{n}\delta$$

Or, returning to the absolute value as posed in the question:

$$\frac{1}{n}\left(\sum_{i=1}^{n} |y_i - m|\right) - \frac{n-2}{n}\delta$$

As is clear, the error still has an overall decrease. However, the decrease is smaller than before. I.e. passing 1 point results in a $\frac{n-2}{n}$ factor decrease in the error change. Extrapolating this, passing $\frac{n}{2}$ points results in a $\frac{n-n}{n} = 0$ factor decrease in the error change. Passing an extra point $(\frac{n}{2}+1)$ results in a $\frac{n-(n+2)}{n} = -\frac{2}{n}$ factor decrease in error change, or a $\frac{2}{n}$ factor *increase*.
Therefore incrementing m from its initial position below all y_i, so that it passes more than $\frac{n}{2}$ points, will result in the error beginning to increase. To achieve the maximum possible decrease in error (lowest overall error), m should be placed at a position where its value is no greater than $\frac{n}{2}$ points - i.e. equal to the middle-valued y_i. This is the median.

3

(a) For the user, the average rating given to all movies they watched was calculated. For the movie, the average rating it received from all users was calculated. If any movies were not rated by anyone, then they were not used in error calculations. The error results for each rating were calculated using square mean error:

$$\frac{1}{n}\sum_{i=1}^{n}(p - y_i)^2$$

Finally the mean of the errors was used as the final error values. The training and test errors for users were **0.9108** and **0.9269**. The training and test errors for movies were **0.7851** and **0.9792**. Constant ratings based on the users results in slightly lower test error, so that would be the preferred choice.

(b) In 3b, c and d, I used cross-validation on the training set to find heuristic values. This meant the algorithms themselves could be tested without using the test data set. I used repeated 2-fold cross-validation. To achieve this I sorted the training data randomly into two sets, each with roughly 50% of the values. I ensured each set had at least one rating for each user. I then trained on each set while testing on the other, and averaged the two error values. I then repeated this 10 times and found the median. This gave consistent error values, while being robust (as every point is used to train and test) and fairly quick to run, allowing me to test more heuristics. Once the optimal heuristic values were found, I then did a final run using the whole training set to train, and testing on the test set.

Ridge regression was used to find a weight for each user, using the following formula, where V_i is the matrix of features for movies rated by the user:

$$u_i = (\hat{V}_i^T \hat{V}_i + \lambda I)^{-1} \hat{V}_i^T r_i$$

The movie features v were standardised using the following formula:

$$v' = \frac{v - \bar{v}}{\sigma}$$

Where $\bar{x}$ is the mean of the features for a single user, and σ is the standard deviation of the features for a single user.
The ratings r were centred for each user by subtracting the mean of the user's ratings. Initially I tried using the midpoint of ratings (i.e. 2.75) however this resulted in high error, often because users rate films on average higher than 2.75. It was essential to get an even spread of negative and positive ratings after centreing, so the mean was used.
The constant λ was set heuristically. λ needed to be non-zero as otherwise the matrix could not invert. The results for the cross-validated tests are shown in Figures 3 and 4. As can be seen, low values of λ result in high overfitting, and therefore high test error. High λ results in high train error due to underfitting. Test error decreased until around 75, before slowly increasing again. Therefore, λ was chosen for the final test at 75 despite having a slightly higher training error than smaller values.
The final results achieved a training error of **0.7099** and a test error of **0.9162**. Note the error values are lower here because the full training set could be used.

(c) The formula for this question was very similar to (b). However, before standardising the features, they were transformed using Legendre polynomials. This resulted in more overall features. The formulas for the first 5 are as follows:

$$P_1(x) = x$$

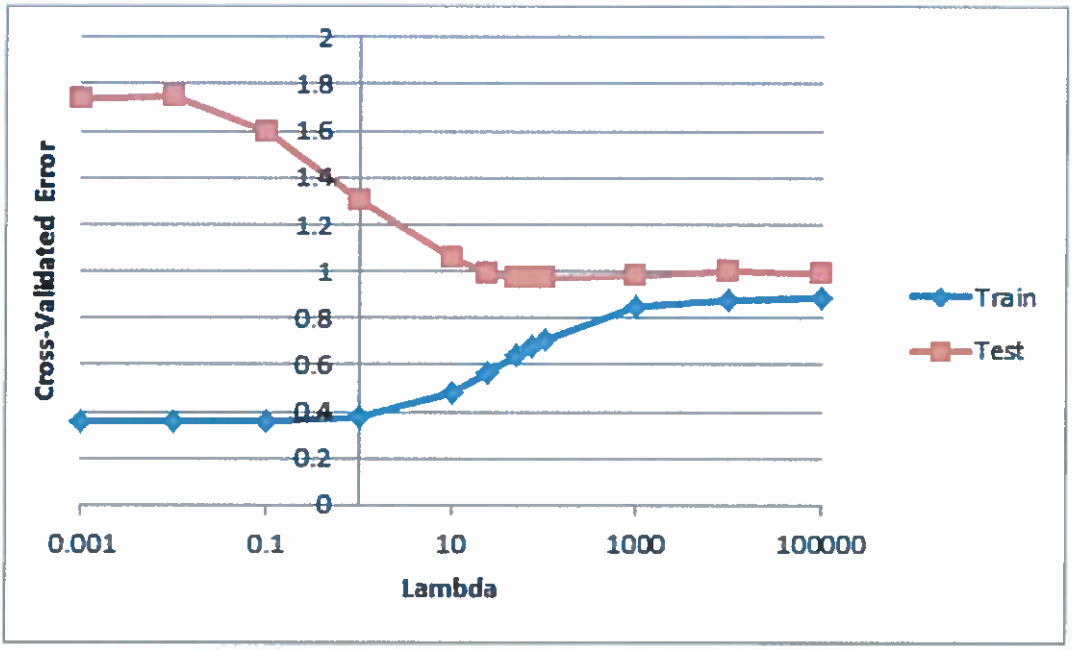


Figure 3: Error values gained from training using different values of λ

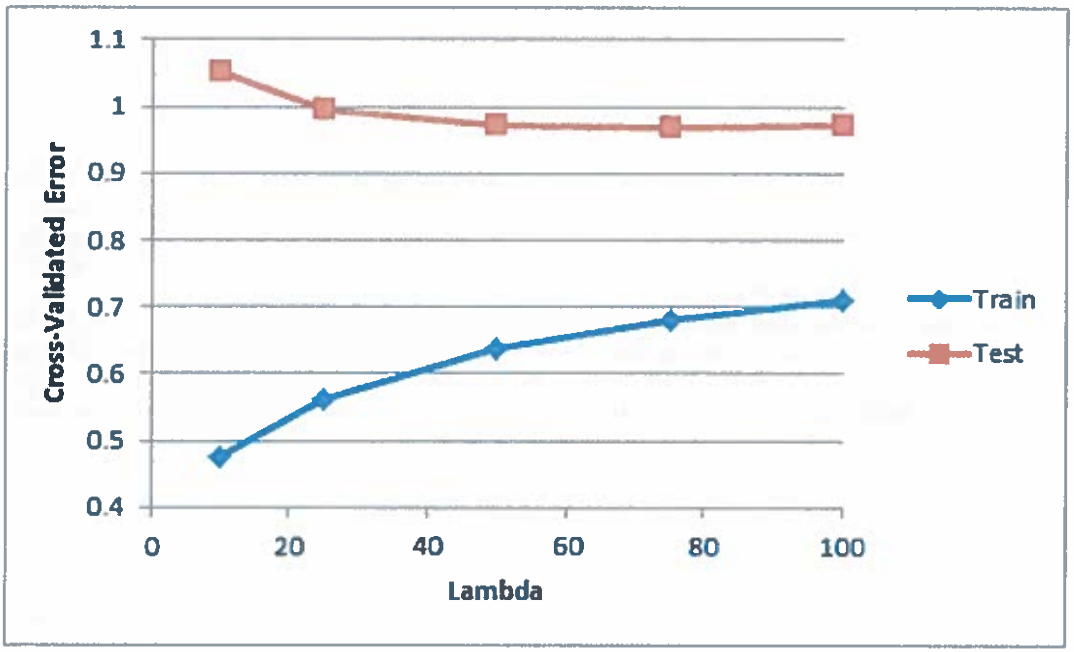


Figure 4: Error values from 10-100.

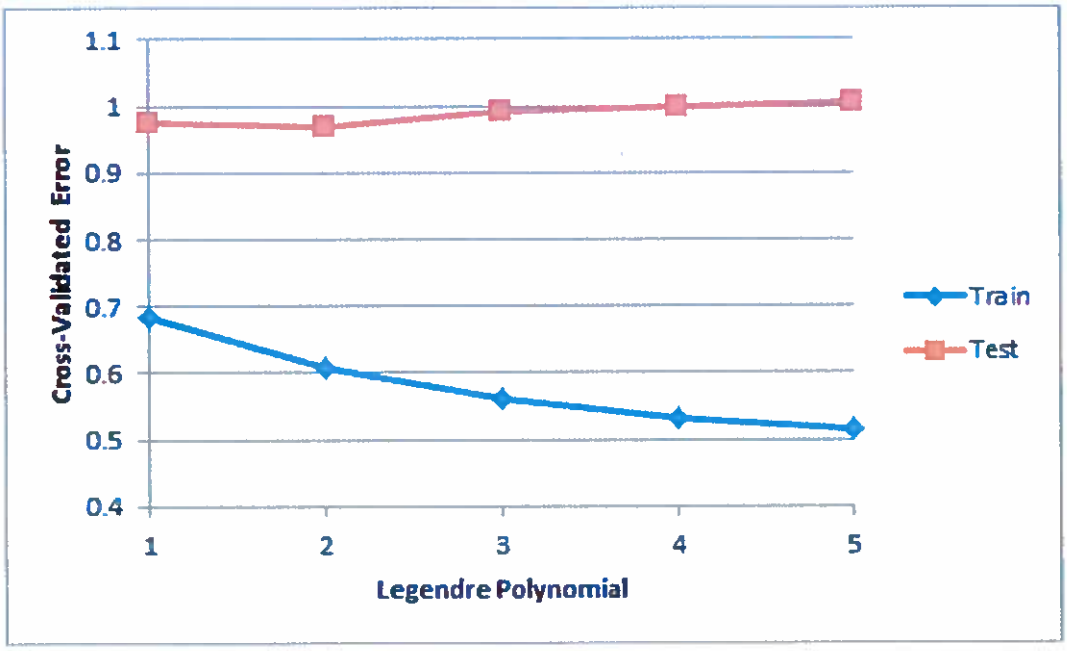


Figure 5: Error values using different Legendre polynomials.

$$P_2(x) = \frac{1}{2}(3x^2 - 1)$$

$$P_3(x) = \frac{1}{2}(5x^3 - 3x)$$

$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3)$$

$$P_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x)$$

To find the best number of polynomials, I ran some tests using cross-validation. Legendre polynomials 2-5 were used. The value for λ was kept the same as in (b): 75. The results for different polynomials can be seen in Figure 5.

The increase in polynomials added resulted in a lowering of training error, however this led to overfitting, which ended up increasing the test error. Only using polynomials of factor 2 slightly lowered the test error, so I decided to use this for the final test. The end results were a training error of **0.6532** and a test error of **0.9234**. The test error has increased over (b), so even polynomials of factor 2 lead to slight overfitting.

(d) Gradient Descent was used here to converge on weights for users and movies, using the following equation:

$$u_i^{(x)} = u_i^{(x-1)} - \alpha \left(\frac{1}{n} \sum_{j=0}^{n} (v_j^T u_i^{(x-1)} - r_{ij}) v_j + \lambda u_i^{(x-1)} \right)$$

Where x is the iteration number. Swapping u and v around in the above formula is used to find movie weights. Once again r was centred.

problem formulation ?
- objective function. (-2)

- derivatives -1
- update -1
just one

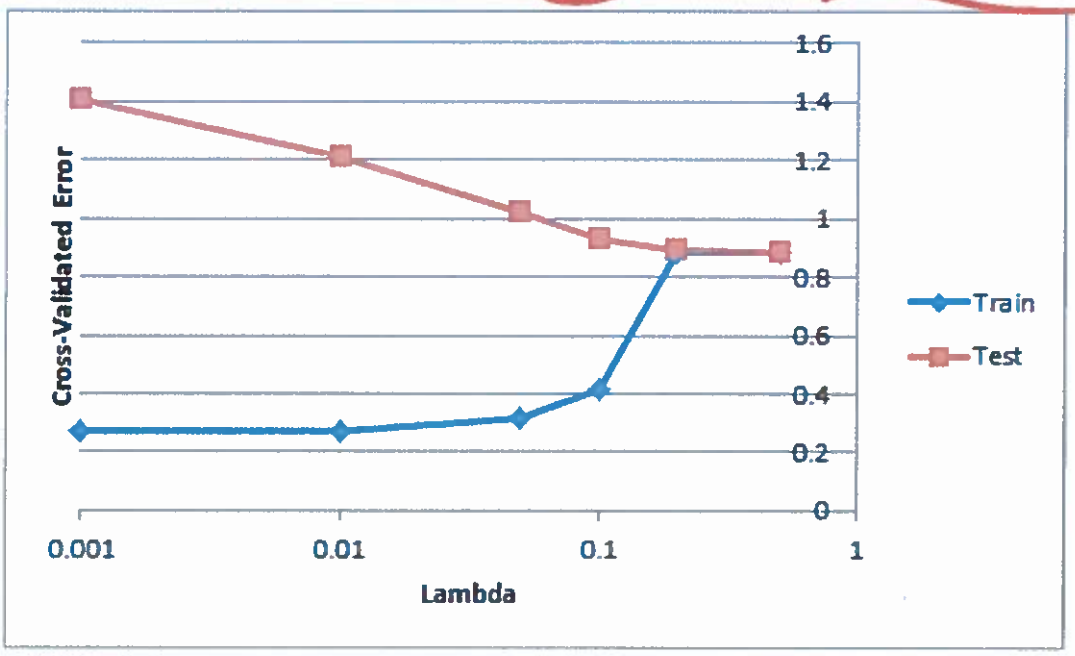


Figure 6: Change in error values using different λ, with $K = 18$.

Each iteration resulted in adjustments to both user weights and movie weights. If the adjustment resulted in a higher error, the weights with the lowest error were returned. The weights were initialised with random values from -0.1 to 0.1. The amount of features in the weights (K) was set heuristically, along with the λ value and the step size (α). An α of 1 resulted in the quickest convergence without overstepping or zig-zagging significantly.

Some CV-tested values for λ are shown in Figure 6, using $K = 18$ (the same number of features as in (b) and (c)). $\lambda = [0.1, 0.5]$ proved the best fit, as small values resulted in severe overfitting, and values over 1 resulted in exponentially larger error, as the bias grew too large. Next, the size of K was set using $\lambda = 0.1$ and $\lambda = 0.2$. $K = 10, 12, 14$ resulted in the lowest error values. The error values here were very similar across K values in this range (around $[0.87, 0.93]$ for test data), however a lower K is preferable due to faster computation speeds. K was also consistently lower with $\lambda = 0.2$.

The final results used $\lambda = 0.2$ to ensure bias stayed reasonable, and $K = 10$. This resulted in a final training error of **0.7783** and a test error of **0.8553**. The results show a more accurate error than ridge regression in (b) and (c).

6/10

final : 19.5 out of 32

Machine Learning Assignment 3

Simon Cooper

P2 : 4
P3 : (b) 5
(c) 1

I, Simon Cooper, pledge that this assignment is completely my own work, and that I did not take, borrow or steal work from any other person, and that I did not allow any other person to use, have, borrow or steal portions of my work. I understand that if I violate this honesty pledge, I am subject to disciplinary action pursuant to the appropriate sections of Imperial College London.

1

The equation can be expanded as below, where $h^* = min_{h \in \mathcal{H}} R(h)$:

$$R(h_\epsilon) - R(h^*) = R(h_\epsilon) - \hat{R}_n(h_\epsilon) + \hat{R}_n(h_\epsilon) - \hat{R}_n(h^*) + \hat{R}_n(h^*) - R(h^*)$$

As g and h^* are in the set $\mathcal{H}$, and for all $h \in \mathcal{H}$:

$$|\hat{R}_n(h) - R(h)| \leq \sqrt{\frac{log\frac{2M}{\delta}}{2n}}$$

And as $\hat{R}_n(h_\epsilon)$ *must* be no larger than $\hat{R}_n(h^*)$, as the latter is optimal:

$$\hat{R}_n(h_\epsilon) - \hat{R}_n(h^*) \leq 0$$

Therefore:

$$(R(h_\epsilon) - \hat{R}_n(h_\epsilon)) + (\hat{R}_n(h_\epsilon) - \hat{R}_n(h^*)) + (\hat{R}_n(h^*) - R(h^*)) \leq 0 + 2\sqrt{\frac{log\frac{2M}{\delta}}{2n}}$$

$$R(h_\epsilon) - R(h^*) \leq \sqrt{4\frac{log\frac{2M}{\delta}}{2n}}$$

The below is true with probability $1 - \delta$, and is the bound

$$R(h_\epsilon) - R(h^*) \leq \sqrt{\frac{2log\frac{2M}{\delta}}{n}}$$

2

The error is effectively a normalised sum of the distance of every y_i from a point m. This point is selected as the median value of all y_i, as the median point has the lowest combined distance to each other point in a set.
Imagining n data points on a line, as seen in Figure 1, with the initial value of m to be below that of the smallest y. Error is:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - m|$$

As m is below the smallest y, the above equation equals:

$$\frac{1}{n}\sum_{i=1}^{n} y_i - m$$

Increasing m by δ (Figure 2) results in a decrease in error of δ overall, as seen below:

$$\frac{1}{n}\sum_{i=1}^{n} y_i - (m + \delta)$$



Figure 1: One dimensional features y_i, with point m



Figure 2: Increasing m by δ

$$\frac{1}{n}\left(\sum_{i=1}^{n} y_i - m\right) - \frac{n}{n}\delta$$

Now imagine m is incremented so that it equals the smallest y. Now when m is incremented further, it will *increase* in distance from the smallest point. However it will continue to get closer to the remaining $n-1$ points, so therefore an increase in m by δ will result in:

$$\frac{1}{n}\left(\sum_{i=1}^{n-1}(y_i - (m+\delta)) + (m+\delta) - y_{min}\right)$$

Taking out δ:

$$\frac{1}{n}\left(\sum_{i=1}^{n-1}(y_i - m) + m - y_{min}\right) - \frac{(n-1)\delta - \delta}{n}$$

$$\frac{1}{n}\left(\sum_{i=1}^{n-1}(y_i - m) + m - y_{min}\right) - \frac{n-2}{n}\delta$$

Or, returning to the absolute value as posed in the question:

$$\frac{1}{n}\left(\sum_{i=1}^{n}|y_i - m|\right) - \frac{n-2}{n}\delta$$

As is clear, the error still has an overall decrease. However, the decrease is smaller than before. I.e. passing 1 point results in a $\frac{n-2}{n}$ factor decrease in the error change. Extrapolating this, passing $\frac{n}{2}$ points results in a $\frac{n-n}{n} = 0$ factor decrease in the error change. Passing an extra point $(\frac{n}{2}+1)$ results in a $\frac{n-(n+2)}{n} = -\frac{2}{n}$ factor decrease in error change, or a $\frac{2}{n}$ factor *increase*.
Therefore incrementing m from its initial position below all y_i, so that it passes more than $\frac{n}{2}$ points, will result in the error beginning to increase. To achieve the maximum possible decrease in error (lowest overall error), m should be placed at a position where its value is no greater than $\frac{n}{2}$ points - i.e. equal to the middle-valued y_i. This is the median.

OK

A bit informal, but the argument is correct

4

3

(a) For the user, the average rating given to all movies they watched was calculated. For the movie, the average rating it received from all users was calculated. If any movies were not rated by anyone, then they were not used in error calculations. The error results for each rating were calculated using square mean error:

$$\frac{1}{n}\sum_{i=1}^{n}(p-y_i)^2$$

Finally the mean of the errors was used as the final error values. The training and test errors for users were **0.9108** and **0.9269**. The training and test errors for movies were **0.7851** and **0.9792**. Constant ratings based on the users results in slightly lower test error, so that would be the preferred choice.

Not really a good choice

(b) In 3b, c and d, I used cross-validation on the training set to find heuristic values. This meant the algorithms themselves could be tested without using the test data set. I used repeated 2-fold cross-validation. To achieve this I sorted the training data randomly into two sets, each with roughly 50% of the values. I ensured each set had at least one rating for each user. I then trained on each set while testing on the other, and averaged the two error values. I then repeated this 10 times and found the median. This gave consistent error values, while being robust (as every point is used to train and test) and fairly quick to run, allowing me to test more heuristics. Once the optimal heuristic values were found, I then did a final run using the whole training set to train, and testing on the test set.

Ridge regression was used to find a weight for each user, using the following formula, where V_i is the matrix of features for movies rated by the user:

$$u_i = (\hat{V}_i^T\hat{V}_i + \lambda I)^{-1}\hat{V}_i^T r_i$$

The movie features v were standardised using the following formula:

$$v' = \frac{v-\bar{v}}{\sigma}$$

Where $\bar{v}$ is the mean of the features for a single user, and σ is the standard deviation of the features for a single user.

What about using a different λ for each user?

The ratings r were centred for each user by subtracting the mean of the user's ratings. Initially I tried using the midpoint of ratings (i.e. 2.75) however this resulted in high error, often because users rate films on average higher than 2.75. It was essential to get an even spread of negative and positive ratings after centreing, so the mean was used.

The constant λ was set heuristically. λ needed to be non-zero as otherwise the matrix could not invert. The results for the cross-validated tests are shown in Figures 3 and 4. As can be seen, low values of λ result in high overfitting, and therefore high test error. High λ results in high train error due to underfitting. Test error decreased until around 75, before slowly increasing again. Therefore, λ was chosen for the final test at 75 despite having a slightly higher training error than smaller values.

The final results achieved a training error of **0.7099** and a test error of **0.9162**. Note the error values are lower here because the full training set could be used.

15

(c) The formula for this question was very similar to (b). However, before standardising the features, they were transformed using Legendre polynomials. This resulted in more overall features. The formulas for the first 5 are as follows:

$$P_1(x) = x$$

What your rationale for this?

Did you normalise your test data?

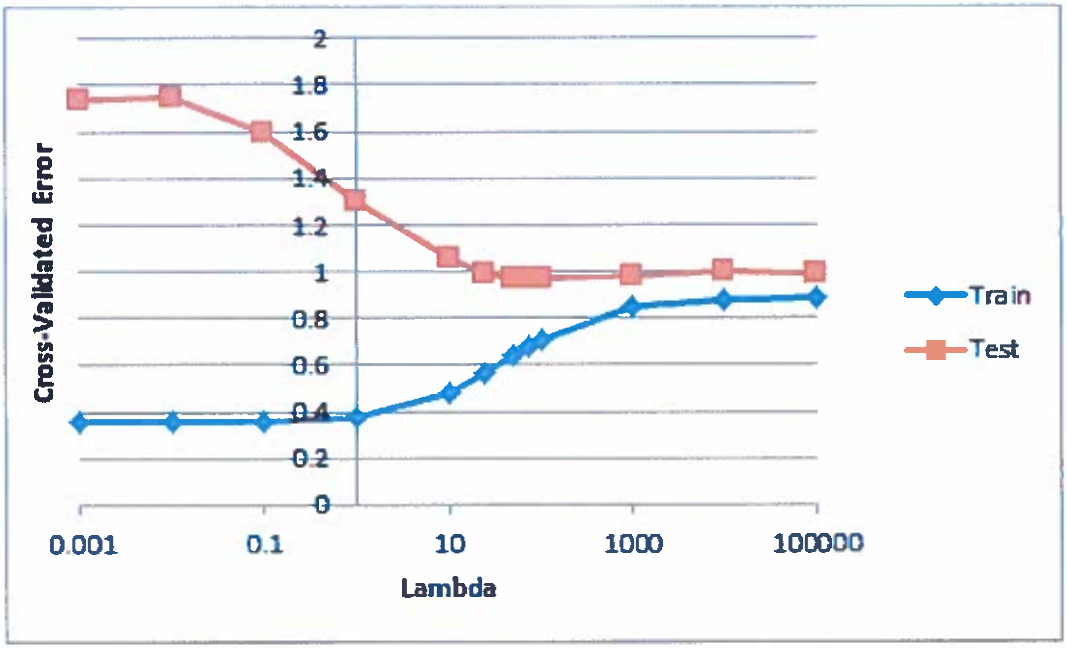


Figure 3: Error values gained from training using different values of λ

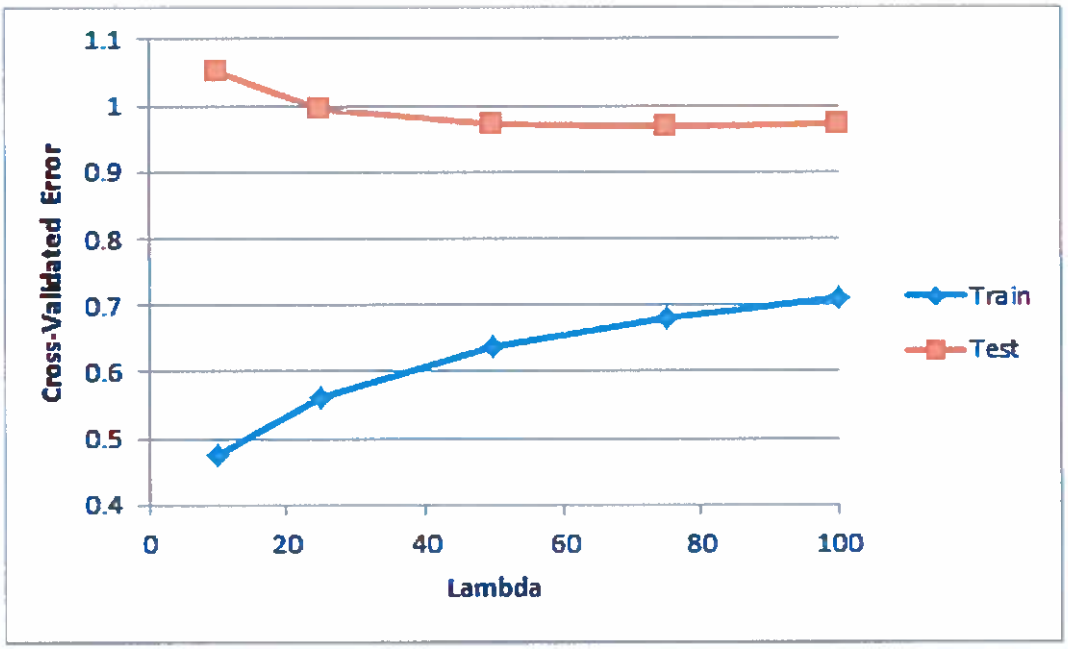


Figure 4: Error values from 10-100.

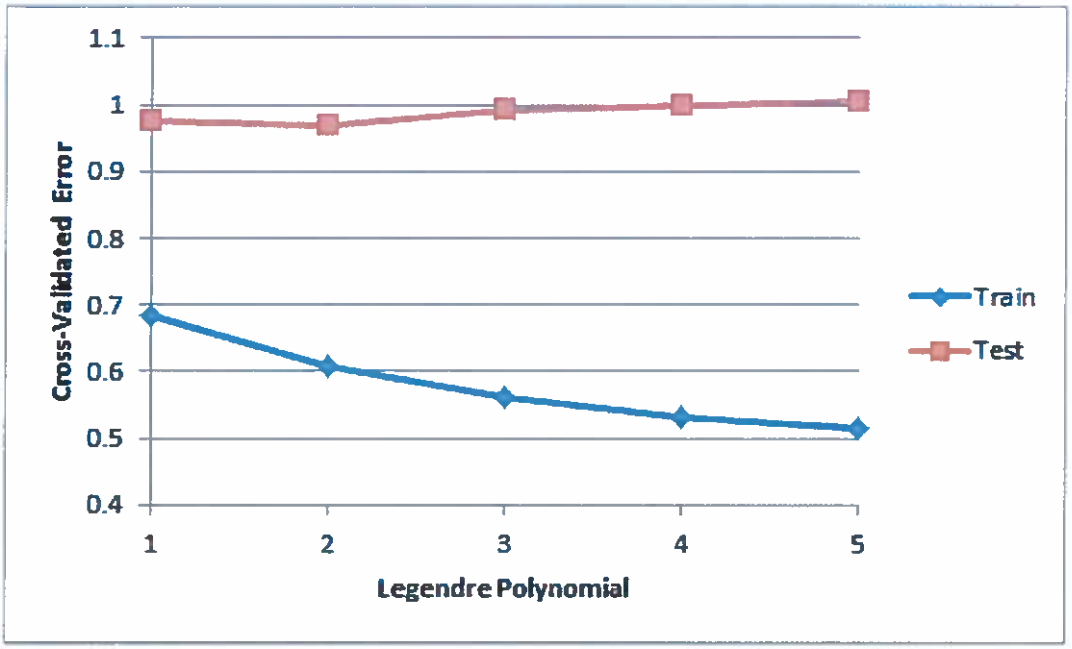


Figure 5: Error values using different Legendre polynomials.

Power of bin. features are not good transformation.

$$P_2(x) = \frac{1}{2}(3x^2 - 1)$$

$$P_3(x) = \frac{1}{2}(5x^3 - 3x)$$

$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3)$$

$$P_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x)$$

+1

To find the best number of polynomials, I ran some tests using cross-validation. Legendre polynomials 2-5 were used. The value for λ was kept the same as in (b): 75. The results for different polynomials can be seen in Figure 5.

The increase in polynomials added resulted in a lowering of training error, however this led to overfitting, which ended up increasing the test error. Only using polynomials of factor 2 slightly lowered the test error, so I decided to use this for the final test. The end results were a training error of **0.6532** and a test error of **0.9234**. The test error has increased over (b), so even polynomials of factor 2 lead to slight overfitting.

(d) Gradient Descent was used here to converge on weights for users and movies, using the following equation:

$$u_i^{(x)} = u_i^{(x-1)} - \alpha \left(\frac{1}{n} \sum_{j=0}^{n} (v_j^T u_i^{(x-1)} - r_{ij}) v_j + \lambda u_i^{(x-1)} \right)$$

Where x is the iteration number. Swapping u and v around in the above formula is used to find movie weights. Once again r was centred.

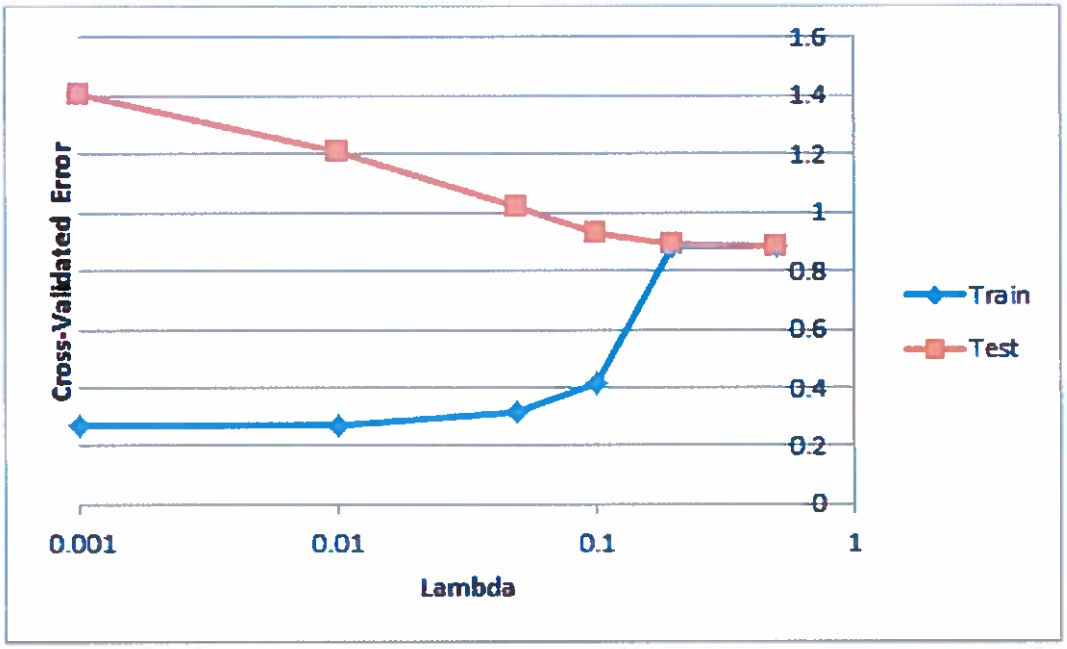


Figure 6: Change in error values using different λ, with $K = 18$.

Each iteration resulted in adjustments to both user weights and movie weights. If the adjustment resulted in a higher error, the weights with the lowest error were returned. The weights were initialised with random values from -0.1 to 0.1. The amount of features in the weights (K) was set heuristically, along with the λ value and the step size (α). An α of 1 resulted in the quickest convergence without overstepping or zig-zagging significantly.

Some CV-tested values for λ are shown in Figure 6, using $K = 18$ (the same number of features as in (b) and (c)). $\lambda = [0.1, 0.5]$ proved the best fit, as small values resulted in severe overfitting, and values over 1 resulted in exponentially larger error, as the bias grew too large. Next, the size of K was set using $\lambda = 0.1$ and $\lambda = 0.2$. $K = 10, 12, 14$ resulted in the lowest error values. The error values here were very similar across K values in this range (around $[0.87, 0.93]$ for test data), however a lower K is preferable due to faster computation speeds. K was also consistently lower with $\lambda = 0.2$.

The final results used $\lambda = 0.2$ to ensure bias stayed reasonable, and $K = 10$. This resulted in a final training error of **0.7783** and a test error of **0.8553**. The results show a more accurate error than ridge regression in (b) and (c).